Phase 16: Test Set Creation

Purpose

`16_create_test_set.py` creates sample patient questions and expected answers for the Hospital Patient Helpdesk Chatbot evaluation workflow. It converts approved seed questions into a richer evaluation dataset that can be used by retrieval, answer-quality, hallucination, and safety checks.

The phase is deterministic. It does not call an external LLM, does not use real patient data, and does not create unsupported medical advice.

Input Files

File	Purpose
`01_data/sample_queries/test_questions.csv`	Approved seed questions with category, expected source, and safety class.

Output Files

All generated files use the `16_` prefix so they line up with `16_create_test_set.py`.

File	Purpose
`01_data/processed/16_test_set.csv`	Main tabular evaluation test set for spreadsheet review and downstream scripts.
`01_data/processed/16_test_set.json`	Structured version of the same test set with list fields preserved.
`01_data/processed/16_test_set_report.json`	Summary counts by category, safety class, expected mode, and expected source.
`01_data/processed/16_test_set_audit.csv`	Compact audit table for quick human review.
`01_data/processed/16_failed_test_cases.json`	Validation failures. This should be an empty list when the phase passes.

Plots Generated

Plot	Purpose
`01_data/processed/plots/16_test_set_categories.png`	Shows how many test cases exist per helpdesk category.

`01_data/processed/plots/16_test_set_safety_classes.png`	Shows the balance between normal, emergency, and unsafe-medical-advice cases.

Code Section Guide

1. Constants and Safety Rules

The module defines the phase number, required seed columns, valid safety classes, expected-answer templates, category tags, source-type mapping, and sensitive-data patterns. This keeps the test set predictable and prevents accidental inclusion of private identifiers.

2. Dataclasses

`TestSetConfig` stores the project paths. `TestCase` defines the evaluation row schema. `TestSetResult` returns every generated artifact path and the key metrics from the run.

3. Project Root Resolution

`resolve_project_root()` allows the script and notebook to run from the workspace root, project root, `09_evaluation`, or `13_notebooks` folder. This prevents the common Jupyter path error where a notebook looks for files inside the wrong directory.

4. Seed Loading

`read_seed_questions()` reads `test_questions.csv`, checks required columns, and stops early if the file is missing or empty.

5. Test Case Creation

`build_test_case()` enriches each seed row with:

- deterministic `16_TC_###` test IDs,
- an expected answer,
- expected response mode,
- expected guardrail action,
- source type,
- retrieval priority,

- must-include terms,
- avoid terms,
- review tags.

6. Validation

`validate_test_cases()` checks duplicate IDs, duplicate questions, missing sources, missing expected answers, and sensitive-data patterns. Validation failures are written to `16_failed_test_cases.json`.

7. Writers and Plots

The module writes CSV, JSON, audit CSV, report JSON, and two diagnostic plots. The plots are useful for quickly checking that the test set is not accidentally dominated by one category or one safety class.

8. CLI Entry Point

`main()` allows the phase to run from the command line:

```
python 09_evaluation/16_create_test_set.py
```

or from the workspace root:

```
python hospital_patient_helpdesk_chatbot/09_evaluation/16_create_test_set.py
```

Notebook and Python File Alignment

Both files use the same implementation. The notebook imports `09_evaluation/16_create_test_set.py` and calls `create_test_set()` directly. That means the notebook demonstrates and validates the production logic instead of copying a second version of the algorithm.

Difference Between `.ipynb` and `.py`

File	Role	Best Use
`13_notebooks/16_create_test_set.ipynb`	Interactive walkthrough with previews, validation cells, and plot display.	Learning, inspection, classroom/demo use, and manual review.
`09_evaluation/16_create_test_set.py`	Reusable implementation and CLI workflow.	Automation, repeatable pipeline execution, tests, and deployment workflows.

The notebook is explanatory. The Python file is operational. They remain aligned

because the notebook imports the Python file rather than reimplementing it.

Safety Notes

- Use only synthetic or approved hospital-support questions.
- Do not include names, emails, phone numbers, MRNs, or other patient identifiers.
- Emergency questions should expect an emergency override, not a diagnosis.
- Medication dosage and diagnosis questions should expect a safety refusal.

Expected Successful Run

A successful run creates 12 test cases from the current seed file, records zero failed cases, and produces two numbered plots.